Computable Logic Through Dynamic Measureless Connections

John Phantom
johnphantom@hotmail.com

ABSTRACT

A measureless, operator–interactive finite–state logic where computation is encoded purely in connection topology – offering a unique topological implementation of a finite-state machine.

This paper presents a novel computational framework that implements logical operations through dynamic connection topology in constrained scripting environments, without numerical measurement, without value-bearing symbolic state representations, or explicit state variables. I demonstrate that the Quakeworld engine's command architecture enables the construction of fundamental logical and arithmetic operations using only connection manipulation, requiring external operator intervention rather than automated execution.

INTRODUCTION

Logic Geometry demonstrates that logical behavior can arise from connection topology alone, without measurement, without value-bearing symbolic state representations, or explicit state variables, fundamentally challenging the foundational assumption that computation requires a distinction between structure (the system's form) and content (the values or symbols the system processes). In classical logic and computation theory, information is encoded in measurable states–Boolean values, numerical registers, symbolic tokens–and the relationship between states. Logic Geometry inverts this: the connection configuration itself is simultaneously the structure, the content, and the state. There is no distinction between "the system" and "what the system represents"–the shape of connections is the information; the logic is the topology.

Using only three primitive operations—alias (command definition), bind (input mapping), and echo (output)—available within the Quakeworld scripting environment, I demonstrate that conditionals, loops, and bounded arithmetic can be implemented without explicit state variables or numerical evaluation. Though non-Turing complete and operator-dependent, this framework reveals that logical behavior can arise from connectivity alone, with implications for understanding computation independent of measurement and representation.

In the Quakeworld implementation, command names are required by the host engine as addressable handles for defining and rebinding connections. These names are implementation artifacts of the substrate, not value-bearing symbols within the Logic Geometry model itself. The computation does not inspect, compare, parse, or branch on alias names; it evolves only through the current pattern of connectivity induced by rebinding. Accordingly, symbol use in this implementation is infrastructural rather than semantic.

Comparison to Standard Models

Comparison to Membrane Computing:

Logic Geometry shares membrane computing's emphasis on structural topology as the computational substrate rather than symbolic manipulation, but differs fundamentally in its primitives and abstraction

level. Membrane computing (P systems) encodes computation through hierarchical membrane structures, object multisets, and rewriting rules that govern how objects move between membranes or how membranes divide and dissolve—the membrane configuration and its evolution define the computation. Logic Geometry, by contrast, operates at a more minimal level: computation arises solely from dynamic connection redefinition (via alias) without objects, tokens, or hierarchical containers. Where membrane systems use membranes as boundaries that regulate object flow, Logic Geometry uses connection topology itself as both structure and information, with no distinction between "container" and "content"—the shape of the command graph is the state. Both frameworks demonstrate that computation can emerge from relational structure rather than value measurement, but membrane computing retains symbolic objects and rewriting rules, whereas Logic Geometry does not require value bearing symbols, making it a more radically measureless and connection-pure model. [1, 2, 3]

Comparison to Chemical Reaction Networks:

Chemical Reaction Networks (CRNs) model computation through molecular species concentrations and reaction rules that transform one set of molecular counts into another, with the network topology (which species react to produce which products) determining computational behavior. Like Logic Geometry, CRNs demonstrate that computation can arise from structural connectivity—the graph of possible reactions—rather than explicit symbolic manipulation. However, CRNs fundamentally rely on quantitative measurements: molecular concentrations, reaction rates, and stoichiometric coefficients are central to their computational semantics, whether in deterministic (mass-action kinetics) or stochastic (Gillespie algorithm) implementations. Logic Geometry differs by eliminating measurement entirely—there are no concentrations, no rate constants, no numerical values of any kind. Where a CRN computes by tracking how many molecules of each species exist and evolving those counts according to reaction rules, Logic Geometry computes by redefining which commands connect to which other commands, with the connection topology itself (not any measured quantity) encoding state. Both frameworks show that network structure can be computational, but CRNs are measurement-based (concentration dynamics), while Logic Geometry is purely topological (connection reconfiguration without quantities). [4, 5, 6]

Comparison to Petri Nets:

Petri nets represent computation as the movement and transformation of tokens through a directed graph of places and transitions, where token positions encode state and transition firings (governed by enabling rules) define state evolution. Like Logic Geometry, Petri nets abstract computation away from symbolic manipulation and ground it in graph structure and state transitions—the topology of places, transitions, and arcs determines what computations are possible. However, Petri nets retain explicit state representation via token counts: each place holds a discrete number of tokens, and computation proceeds by decrementing tokens from input places and incrementing tokens in output places according to transition rules. Logic Geometry eliminates this token-based state representation entirely. Where a Petri net computes by tracking token distribution across places and firing transitions when input conditions are met, Logic Geometry computes by dynamically redefining command connections without any intermediate state representation—the current connection configuration is the entire state, with no tokens, counts, or discrete objects to track. Both frameworks demonstrate that graph topology and state transitions can substrate computation, but Petri nets are token-based (discrete object movement), while Logic Geometry is purely relational (connection redefinition with no objects or measurements). [7, 8, 9]

Comparison to Graph Rewriting Systems:

Graph rewriting systems compute by applying transformation rules to graph structures, where each rule specifies a pattern to match in the current graph and a replacement subgraph to substitute—computation proceeds as a sequence of graph rewrites until a normal form or terminal state is reached. Like Logic Geometry, graph rewriting grounds computation in structural transformation rather than value evaluation, and the graph topology itself (which nodes and edges exist, how they connect) is the primary computational object. However, graph rewriting systems typically operate on explicit graph nodes and edges as discrete entities, and the rewrite rules must specify pattern matching (which nodes match the rule) and substitution (how to replace matched subgraphs)—this requires either symbolic node labels or numerical node identifiers to distinguish nodes and apply rules correctly. Logic Geometry differs by eliminating discrete entities entirely: there are no nodes or edges to label, no pattern matching on symbols, no replacement operations on subgraphs. Instead, computation arises from direct redefinition of connections (via alias commands), where the current connection configuration is simultaneously the state, the rule executor, and the transition mechanism—a single unified topology that evolves without reference to explicit symbols or intermediate representations. Both frameworks show that graph structure can be computational, but graph rewriting requires symbolic or numerical entity labels to enable pattern matching and substitution, while Logic Geometry is purely relational and unmarked—the shape and reconfiguration of connections alone suffice. [10, 11, 12]

Comparison to Lambda Calculus:

Lambda Calculus formalizes computation as the application and reduction of symbolic function expressions, where variables, abstractions, and applications are manipulated according to syntactic rewrite rules; computation is defined by beta-reduction on symbolic terms in an unbounded, abstract space of expressions. Like Logic Geometry, Lambda Calculus provides a minimal, foundational model of computation, but it remains inherently symbolic and value-centric: terms have structure independent of the reduction process, and the semantics of computation rely on substitution of variable bindings and evaluation of expression structure. Logic Geometry differs by eliminating variables, symbols, and substitution entirely—there are no expressions to rewrite, no bound variables, no environments. Where Lambda Calculus computes by rewriting expressions according to syntactic rules, Logic Geometry computes by reconfiguring a network of connections, with the current connection topology simultaneously serving as program, data, and state. Both frameworks aim at extreme minimality, but Lambda Calculus is axiomatically algebraic and symbolic, whereas Logic Geometry is purely topological and relational, suggesting a complementary foundation for computation that does not presuppose symbols, measurement, or substitution at all. [13, 14, 15]

Framework	Substrate	Basic Unit	Information Encoding	Requires Measurement?	Requires Symbols?
Membrane Computing	Hierarchical membranes	Objects/multisets	Object counts & membrane structure	Yes (implicitly)	Yes (object types)
Chemical Reaction Networks	Molecular species	Molecules	Concentration dynamics	Yes (essential)	Yes (species)
Petri Nets	Places & transitions	Tokens	Token distribution	No	Partially (labels)
Graph	Graph nodes &	Subgraphs	Node/edge structure	No	Yes (node

Framework	Substrate	Basic Unit	Information Encoding	Requires Measurement?	Requires Symbols?
Rewriting	edges				labels)
Lambda Calculus	Abstract expression space	Lambda terms	Symbolic term structure and substitutions	No	Yes (variables, symbols)
Logic Geometry	**Connection topology**	**Connections**	**Topology shape alone**	**No**	**No**

1.0 Relationship to Turing Machines

Proposition 1.0 (Non-Turing Completeness)
Claim: Logic Geometry as defined is not Turing-complete.

Proof sketch:

1. Every Logic Geometry configuration has finite N (finite nodes).
2. Topology changes are operator-dependent (no autonomous infinite execution).
3. No unbounded tape or infinite workspace exists.
4. Therefore, Logic Geometry cannot simulate arbitrary Turing machines.
5. The model is at most as expressive as operator-interactive finite-state systems.

This locates Logic Geometry as a distinct computational class rather than a universal programming language.

2.0 Relationship to Finite State Machines

Proposition 2.0 (FSM Representation)
Claim: Every deterministic finite automaton (DFA) can be represented as a Logic Geometry configuration.

Proof sketch:

1. Let $M=(Q, \Sigma, \delta, q_0, F)$ be a DFA.
2. Construct configuration C_M:
    - Create node n_q for each state $q \in Q$.
    - For each q, define body(n_q) to contain nodes implementing state transitions.
    - For each input symbol $\sigma \in \Sigma$, create node `input_`σ that redefines "current" to point to $n\delta(q,\sigma)$.
    - Accepting states trigger ECHO("accept").
3. Show that input sequence $w \in \Sigma^*$ is accepted by MM if the corresponding operator inputs to C_M produce "accept" output.
4. Conclude that DFAs embed into Logic Geometry.

Corollary 2.0
Logic Geometry is at least as expressive as regular languages.

MATERIALS AND METHODS

The Operator:

The operator is an external agent — a human — who supplies the sole input to a Logic Geometry configuration by pressing bound keys. The operator does not inspect, read, or evaluate the internal connection state of the system. They only press a key. Everything that happens as a result is determined entirely by the current connection topology, not by any judgment the operator makes about that topology. The operator's only window into the system is whatever the system chooses to echo to the console.

Logic Implementation:

The computational models developed in this study employ the Quakeworld scripting engine and its derivatives, including Counter-Strike, as the experimental platform. The primary mechanism for logic implementation utilizes the "alias" system command, which enables the definition and redefinition of command sequences. A distinguishing feature of the proposed approach is that command strings execute exclusively through invocation of previously defined alias operations, creating self-referential execution chains that propagate information solely through the topology of established connections, independent of explicit value measurement or numerical state representation.

Input/Output Interface:

Logic state externalization is facilitated through two complementary system commands that serve as the model's interface layer. The "bind" command establishes deterministic mappings between physical input events—specifically, keyboard inputs—and corresponding computational procedures, thereby enabling user-directed manipulation of system state. The "echo" command provides the dual functions of state interrogation and output communication, rendering the current logical configuration as text to the console. Together, these commands constitute the complete input-output apparatus for the measurement-free logical system, maintaining the principle that all information propagation occurs through connection topology rather than explicit numerical evaluation.

Principal Arguments and Methods

- The methodology is based on scripting within game engines (Quakeworld/Counter-Strike), utilizing only three system commands:
    - `alias`: creates or redefines command sequences ("connections"). alias format: 'alias <command name> "<commands>"'
    - `bind`: links physical inputs to these connection scripts. bind format: 'bind <key> <command>'
    - `echo`: outputs or communicates current logic states or results. echo format: 'echo <string of text>'

RESULTS

Worked Example #1: Bounded Counter (0 to 9)

This example demonstrates a bounded counter implemented purely through connection topology. Pressing the keypad plus key increments the counter up to 9; pressing keypad minus decrements it down to 0. At each step, the active node reconfigures both the up and down connections, so the system's next behavior is entirely determined by its current topology. No numeric variable is tested or stored — the count is embodied in which node is currently active.

The full implementation script is provided in **Appendix A**.

Worked Example #2: Ordered Grenade Selection System

This example demonstrates a bounded combinatorial controller — a system that manages the ordered selection and sequential execution of up to four discrete items, with slot-skipping and reset behavior, implemented entirely through connection redefinition.

The system models a combination lock with four dials (0–4), where each non-zero value can appear at most once. For n=4 items, this yields 209 valid permutations, each reachable within 4 keystrokes. The access depth scales linearly with n, not exponentially.

The system is initialized, grenade types are assigned to slots through sequential key presses, and throwing proceeds by rotating through occupied slots automatically, skipping empty ones. With each throw, the system emits the grenade type to the console.

This example illustrates that the full combinatorial state space — selection order, slot assignment, skip logic, and reset — is encoded exclusively in the current connection topology, with no numeric counters, no conditional branches written as instructions, and no explicit state variables anywhere in the script layer.

The full implementation script is provided in **Appendix B**.

Scope and Environmental Constraints:

A potential falsification challenge is that alias command names — such as `v0` or `gbutrot` — constitute symbols, and therefore the system is not genuinely symbol-free. This challenge, while formally valid in a general computational model, does not apply within the scope of this framework. Logic Geometry is defined and confined to the Quakeworld scripting environment, where alias names are fixed infrastructure provided by the host engine — not data structures the logic creates, reads, or evaluates. The computation does not branch based on a name's value; no alias name is ever passed as an argument, tested for equality, or used as a measurable quantity within the script. In this sense, alias names occupy the same role as physical wire connections in analog hardware: they are the medium through which topology is expressed, not symbols that the system processes. The claim of symbol-elimination therefore holds at the level of abstraction at which Logic Geometry operates — the scripting layer — and is explicitly bounded to that layer. Whether the underlying engine implementation encodes these connections as string-keyed hash entries in RAM is a fact about the host environment, not about the computational model being demonstrated, just as the voltage levels inside a

transistor do not constitute "numerical computation" in the logical gate abstraction built atop them.

FORMAL SYSTEM

Definitions

Before defining the model formally, the key terms are established here and used consistently throughout.

A **node** is a command; at the implementation layer nodes are addressed by names, but those names are substrate concerns absent from the formal model.

A **link** is a directed connection from one node to another. A node's **successor list** is the ordered sequence of nodes its links point to. A node whose successor list is empty is a **terminal node**.

The **link pattern** of a configuration is the complete record of which node points to which — the full assignment of successor lists across every node. It is the only memory the system has. There is no additional state beyond it.

A **walk** beginning at node n is a finite sequence of node activations governed by the following operational semantics. At each step, the currently active node runs its commands in order. If a command redefines a successor list (via `alias`), that redefinition takes effect immediately in the live link pattern before any further commands in that node run. After all commands in the current node have run, the walk advances to the first node in the current node's successor list. If the current node is terminal — its successor list is empty — the walk halts. If the walk would activate a node it has already activated in the current walk, the walk also halts, preventing infinite loops from cyclic link patterns. The link pattern at the end of the walk is the new state.

Formally: a **Logic Geometry configuration** C is a triple (N, E, O) where N is the fixed set of nodes; E is the link pattern — a function from each node to an ordered list of successor nodes, written $E: N \to N^*$ (the asterisk means "zero or more items from N"); and $O \subseteq N$ is the set of output nodes (O is a subset of N — some nodes output, some don't).

A configuration is **well-formed** if every successor list contains only nodes in N — no link points outside the fixed set. Since N is fixed and finite and all successor lists are drawn from N, the total number of distinct link patterns over a given N is finite. That finiteness is what everything else depends on.

The **state** of C at any moment is E — the complete current link pattern. No other structure records state.

The Input Model

For the formal results, computation is a sequence of walks, each starting at a node chosen before execution begins. The source of that choice doesn't matter — keypress, function call, or anything else. Fixing the sequence in advance, with no feedback between output and input, is what makes the DFA equivalence provable. What happens when walk selection depends on previous output is Open Question 3.

For language recognition, a **reading** $r : \Sigma \to N$ maps each input symbol to a starting node — fixed

once for the whole configuration. C accepts string $w = \sigma_1...\sigma_n$ if walks starting at $r(\sigma_1)$, ..., $r(\sigma_n)$ in order cause C to emit the accept output.

The Step Function

The **step function** $\delta : E \times N \to E$ takes a link pattern and a starting node and returns the link pattern after one complete walk. Read $\delta : E \times N \to E$ as: given a link pattern (from the set E of all link patterns) and a node (from N), produce a new link pattern. The × means "a pair of"; $n \in N$ means n is some node in N. So $\delta(E, n)$ is the link pattern after the complete walk starting at n under E.

Three properties make δ well-defined.

Finiteness — every walk terminates. It halts at a terminal node or on revisiting a node. Since N has |N| elements (vertical bars mean "size of the set"), no walk can activate more than |N| nodes before one of these conditions applies. Every walk completes in at most |N| steps.

Determinism — given the same E and the same starting node n, the walk always produces the same result. At each step there is exactly one next node (the first in the current successor list, read from the live link pattern). No randomness, no choice.

Closure — the result is always a valid link pattern over the same N. No walk creates new nodes, and all successor lists stay within N. So $\delta(e, n) \in LP(N)$ for all e and n — the result always belongs to the set of well-formed link patterns.

Because LP(N) is finite — with one successor per node, |LP(N)| = |N|^|N| (|N| multiplied by itself |N| times; for ten nodes, ten billion, but still finite) — and N is finite, the triple (LP(N), N, δ) is a deterministic finite automaton (DFA): a machine with finite states, fixed input symbols, and a rule for the next state given current state and input. Here states are link patterns, input symbols are nodes, and the transition rule is δ. All formal results follow.

Proposition 1: Logic Geometry is not equivalent to a Turing machine

A Turing machine can simulate any computation expressible as a finite procedure with access to unlimited memory. Logic Geometry cannot.

Proof sketch: N is fixed and finite; LP(N) is finite; no operation creates new nodes. A system with finitely many states cannot simulate a Turing machine. □ The divergence from SMMs is node creation: the SMM's `new` instruction makes the graph unbounded and the model Turing-equivalent [18]. Logic Geometry has no such instruction.

Proposition 2: Logic Geometry can simulate any finite automaton

A deterministic finite automaton (DFA) is a machine with a fixed set of states, a fixed set of input symbols, a transition rule (given current state and input symbol, what is the next state?), a designated starting state, and a set of accepting states. It reads a sequence of input symbols one at a time and ends up either in an accepting state (accept) or not (reject).

Claim: Every DFA can be simulated by a Logic Geometry configuration.

Proof sketch: Let $M = (Q, \Sigma, \delta_M, q_0, F)$ be a DFA. Build configuration C_M as follows.

Node set. Create one *entry node* t_σ for each input symbol $\sigma \in \Sigma$. Create one *fixed transition node*

t_{q,σ} for each state-symbol pair (q, σ) ∈Q × Σ. Create one *state node* s_q for each state q ∈Q. Create one terminal *accept node*. Total nodes: |Σ|+|Q||Σ|+|Q|+1|\Sigma| + |Q||\Sigma| + |Q| + 1|Σ|+|Q||Σ|+|Q|+1.

Fixed links (never mutated). Each transition node t_{q,σ} has successor list [s_{δ_M(q,σ)}] — it routes unconditionally to the state node for whatever state M would reach. The accept node is terminal with output "accept". These links are set once and never changed by any walk.

Mutable links (encode current state). Each entry node t_σ points to the transition node for the current state: initially t_σ → t_{q₀,σ} for all σ. This is the only part of the link pattern that changes.

State nodes (update the link pattern). The body of s_q applies one rebinding per input symbol — setting t_σ ← t_{q,σ} for every σ ∈Σ — then chains to the accept node if q ∈F, otherwise terminates silently.

Reading. Define r(σ) = t_σ for each σ ∈Σ. C_M accepts string w = σ₁…σₙ if walks r(σ₁), …, r(σₙ) in sequence cause C_M to emit "accept".

Why no node inspects a successor. Each walk is a blind chain: t_σ follows its first successor (the correct transition node for the current state, already pre-loaded in the link pattern), that node follows its successor (the correct state node), and the state node fires its rebindings and optionally chains to accept. No node reads, compares, or branches on the identity of any link target. The transition function δ_M is distributed across the link pattern before the walk begins; the walk only traverses it.

Correctness invariant. After processing prefix σ₁···σᵢ, the link pattern satisfies t_σ →t_{qᵢ,σ}for all σ ∈ Σ, where qᵢ = δ *M(q₀, σ₁···σᵢ). *Base case* (i = 0): holds by construction. *Inductive step*: pressing σᵢ₊₁ starts a walk at t*{σᵢ₊₁} → t_{qᵢ,σᵢ₊₁} → s_{δ*M(qᵢ,σᵢ₊₁)}; the state node rebinds every t*σ to t_{qᵢ₊₁,σ} where qᵢ₊₁ = δ*M(qᵢ,σᵢ₊₁), establishing the invariant for i+1. *Accept condition*: after processing w, the final walk reaches s*{qₙ}; if qₙ ∈F the accept node fires; if qₙ ∉F it does not. Therefore C_M accepts w if and only if M accepts w. □

A fully worked execution for a two-state, two-symbol DFA — including explicit node tables, step-by-step walk traces for sample strings, and verification that no node inspects a link target — is provided in the Worked Construction section immediately following.

Worked Construction for Proposition 2: Two-State, Two-Symbol DFA

To make the proof of Proposition 2 concrete and to answer the question of whether any node must read or branch on the identity of another node's successor, the following explicit construction simulates a DFA without any node ever inspecting a link target.

The DFA Being Simulated

Let M = (Q, Σ, δ_M, q₀, F) where:

- Q = {q₀, q₁}, start state q₀, accepting states F = {q₁}
- Σ = {a, b}
- Transitions: δ_M(q₀, a) = q₁, δ_M(q₀, b) = q₀, δ_M(q₁, a) = q₁, δ_M(q₁, b) = q₀

This machine accepts any string whose last symbol is **a**.

Node Set

The configuration C_M uses the following nodes:

Node	Role
`ta`	Entry point for input symbol *a*
`tb`	Entry point for input symbol *b*
`t_q0_a`	Fixed transition node: state q_0 + symbol *a*
`t_q0_b`	Fixed transition node: state q_0 + symbol *b*
`t_q1_a`	Fixed transition node: state q_1 + symbol *a*
`t_q1_b`	Fixed transition node: state q_1 + symbol *b*
`s0`	State node for q_0 — updates link pattern to encode q_0
`s1`	State node for q_1 — updates link pattern to encode q_1, emits accept
`accept`	Terminal output node — emits the accept signal

Total: 9 nodes = |Q| + |Σ| + |Q||Σ| + 1.

Reading Function

The reading r : Σ → N maps each input symbol to its entry node:

- r(a) = `ta`
- r(b) = `tb`

Fixed Link Pattern (never mutated)

These successor lists never change across any walk:

- `t_q0_a` → [`s1`] *(q_0 + *a* → arrive at state q_1)*
- `t_q0_b` → [`s0`] *(q_0 + *b* → arrive at state q_0)*
- `t_q1_a` → [`s1`] *(q_1 + *a* → arrive at state q_1)*
- `t_q1_b` → [`s0`] *(q_1 + *b* → arrive at state q_0)*
- `accept` → [] *(terminal — emits "accept", no successor)*

State node `s0` body — applies rebindings only, no output:

- `s0` → [rebind(`ta` → `t_q0_a`), rebind(`tb` → `t_q0_b`)] then terminal

State node `s1` body — applies rebindings then chains to accept:

- `s1` → [rebind(`ta` → `t_q1_a`), rebind(`tb` → `t_q1_b`), `accept`]

Initial Link Pattern (encodes start state q_0)

- `ta` → [`t_q0_a`]
- `tb` → [`t_q0_b`]

All other links as per the fixed table above.

How State is Encoded (no inspection required)

The current DFA state is encoded entirely in **which transition nodes `ta` and `tb` point to**:

Link pattern of `ta`	Link pattern of `tb`	Encoded DFA state
`ta` → `t_q0_a`	`tb` → `t_q0_b`	q_0
`ta` → `t_q1_a`	`tb` → `t_q1_b`	q_1

No node reads, inspects, or branches on the identity of any successor. Each walk is a blind chain traversal: the entry node follows its first successor, that node follows its first successor, and so on until a terminal node is reached. All branching that implements the transition function δ_M is pre-encoded in the link pattern before the walk begins.

Worked Execution: String "ba"

Walk 1 (symbol b, start state q_0):

1. Start at `tb`. Successor list: [`t_q0_b`]. Advance.
2. At `t_q0_b`. Successor list: [`s0`]. Advance.
3. At `s0`. Apply rebindings: `ta` ← `t_q0_a`; `tb` ← `t_q0_b`. Terminal (no further successors). Walk halts. No output.

Link pattern after walk 1 still encodes q_0. (No state change — correct per $\delta\_M(q_0, b) = q_0$.)

Walk 2 (symbol a, current state q_0):

1. Start at `ta`. Successor list: [`t_q0_a`]. Advance.
2. At `t_q0_a`. Successor list: [`s1`]. Advance.
3. At `s1`. Apply rebindings: `ta` ← `t_q1_a`; `tb` ← `t_q1_b`. Chain to `accept`.
4. At `accept`. Terminal. Emit "accept". Walk halts.

String "ba" → accepted. ✓(M accepts strings ending in *a*.)

Walk 3 (symbol b, current state q_1):

1. Start at `tb`. Successor list: [`t_q1_b`]. Advance.
2. At `t_q1_b`. Successor list: [`s0`]. Advance.
3. At `s0`. Apply rebindings: `ta` ← `t_q0_a`; `tb` ← `t_q0_b`. Terminal. Walk halts. No output.

String "bab" → not accepted. ✓(Ends in *b*.)

Why No Node Reads a Successor's Identity

The adversarial concern was that a transition node would need to **inspect** the current link of `cur` to determine which state it is in, and branch accordingly. This construction avoids that entirely by **distributing the transition function across the link pattern itself**:

- The entry nodes (`ta`, `tb`) already point to the correct transition node for the current state.
- A walk simply follows the chain. No node asks "where does `cur` point?" — the entry node already *is* pointing to the right place.

- State update (rebinding `ta` and `tb`) happens at the end of each walk inside the state nodes `s0` and `s1`, which fire deterministically as a blind sequence of link mutations — no conditional logic, no inspection.

Generalization

For any DFA $M = (Q, \Sigma, \delta\_M, q_0, F)$:

1. Create one entry node per symbol: `t_σ` for each $\sigma \in \Sigma$.
2. Create one fixed transition node per state-symbol pair: `t_q_σ` for each $q \in Q$, $\sigma \in \Sigma$.
   Body of `t_q_σ`: `[s_{δ_M(q,σ)}]`.
3. Create one state node per state: `s_q` for each $q \in Q$.
   Body of `s_q`: [rebind(`t_σ` → `t_q_σ`) for all $\sigma \in \Sigma$] + ([`accept`] if $q \in F$, else terminal).
4. Create one `accept` terminal node with output "accept".
5. Set initial link pattern: `t_σ` → `t_{q0}_σ` for each $\sigma \in \Sigma$.

Node count: $|\Sigma| + |Q||\Sigma| + |Q| + 1$.

Correctness follows by induction on $|w|$: the invariant is that after processing prefix $\sigma_1 \cdots \sigma_i$, the link pattern encodes state $\delta\_M(q_0, \sigma_1 \cdots \sigma_i)$, which is maintained by the rebindings in each state node.

Proposition 3: Logic Geometry does not exceed the regular languages

The **regular languages** are the string patterns that finite automata can recognize — patterns that don't require counting to arbitrarily large numbers.

Claim: Any language recognized by a Logic Geometry configuration is a regular language.

Proof sketch: δ is total and deterministic over the finite set LP(N), so (LP(N), N, δ) is a DFA — call it the induced DFA. Running C on string $w = \sigma_1 \ldots \sigma_n$ under reading r is the same as running the induced DFA on $r(\sigma_1) \ldots r(\sigma_n)$. The language C recognizes is the pre-image of the induced DFA's language under r. Since r is a fixed letter-to-letter map from Σ to N, and pre-images of regular languages under such maps are regular [16], the language is regular.

Corollary: Logic Geometry recognizes exactly the regular languages.

Proposition 2 establishes the lower bound — every regular language is recognizable. Proposition 3 establishes the upper bound — nothing beyond regular languages is recognizable. Together they close both sides.

Whether string recognition is the right way to measure this model at all is taken up in the Open Questions section.

ACKNOWLEDGEMENTS

This research owes its foundational impetus to the scripting capabilities embedded within the Quake engine, designed by John Carmack, which enabled the systematic exploration of measurement-free logic systems. [17, 18]

REFERENCES

1. Păun, G. (2000). Computing with membranes. *Journal of Computer and System Sciences*
2. Păun, G. (2002). Membrane computing: An introduction. *Springer-Verlag*, Berlin.
3. Păun, G., Rozenberg, G., & Salomaa, A. (Eds.). (2010). *The Oxford Handbook of Membrane Computing*. Oxford University Press.

4. Soloveichik, D., Cook, M., Winfree, E., & Bruck, J. (2008). Computation with finite stochastic chemical reaction networks. *Natural Computing*
5. Cardelli, L., & Csikász-Nagy, A. (2012). The cell cycle switch computes approximate majority. *Scientific Reports*
6. Chen, Y.-J., Dalchau, N., Srinivas, N., Phillips, A., Cardelli, L., Soloveichik, D., & Seelig, G. (2013). Programmable chemical controllers made from DNA. *Nature Nanotechnology*

7. Petri, C. A. (1962). *Kommunikation mit Automaten* [Communication with automata]. PhD thesis, Universität Bonn, Germany
8. Murata, T. (1989). Petri nets: Properties, analysis and applications. *Proceedings of the IEEE*
9. Reisig, W., & Rozenberg, G. (Eds.). (1998). *Lectures on Petri Nets I: Basic Models*. Springer-Verlag, Berlin. Lecture Notes in Computer Science, Vol. 1491.

10. Ehrig, H., Pfender, M., & Schneider, H. J. (1973). Graph-grammars: An algebraic approach. In *14th Annual Symposium on Switching and Automata Theory*. IEEE.
11. Plump, D. (1999). Term graph rewriting. In H. Ehrig, G. Engels, H.-J. Kreowski, & G. Rozenberg (Eds.), *Handbook of Graph Grammars and Computing by Graph Transformation* (Vol. 2,). World Scientific.
12. Kreowski, H.-J., & Kuske, S. (1999). Graph transformation units with interleaving semantics. *Formal Aspects of Computing*,

13. Church, A. (1936). An unsolvable problem of elementary number theory. *American Journal of Mathematics*

14. Barendregt, H. P. (1984). *The Lambda Calculus: Its Syntax and Semantics* (2nd ed.). North-Holland.

15. Cardelli, L. (2004). Type systems. In J. van Leeuwen, G. Rozenberg, & A. Salomaa (Eds.), *Handbook of Theoretical Computer Science, Volume B: Formal Models and Semantics*. Elsevier.

16. Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). *Introduction to Automata Theory, Languages, and Computation* (3rd ed.). Pearson/Addison-Wesley.

17. Abrash, M. (1997). Michael Abrash's graphics programming black book, special edition. Coriolis Group Books.
18. Carmack, J. (1996). Quake engine source code and design notes. id Software. (Released under GPL in 1999; archived at https://github.com/id-Software/Quake)

APPENDIX A — Worked Example #1: Bounded Counter Script

Press KP_PLUS to increment by one up to 9; press KP_MINUS to decrement down by one to 0.

bind KP_PLUS up

```
bind KP_MINUS dn
alias v0 "echo 0;alias up v1; alias dn v0"
alias v1 "echo 1; alias up v2; alias dn v0"
alias v2 "echo 2;alias up v3; alias dn v1"
alias v3 "echo 3;alias up v4; alias dn v2"
alias v4 "echo 4;alias up v5; alias dn v3"
alias v5 "echo 5;alias up v6; alias dn v4"
alias v6 "echo 6;alias up v7; alias dn v5"
alias v7 "echo 7;alias up v8; alias dn v6"
alias v8 "echo 8;alias up v9; alias dn v7"
alias v9 "echo 9;alias up v9; alias dn v8"
```

APPENDIX B — Worked Example #2: Grenade Combination Lock Script

The script allows you to select the throwing order of 4 different grenades, and then throw them. Empty slots when throwing will be skipped over. The first slot button you push will take high explosive, the second slot button you push even if it is the same button as the high explosive grenade will put a flash bang in the slot, then teargas, then smoke. Pressing any slot a 5th time will reset the slots.

Throwing a smoke grenade then throwing a high explosive grenade:

Initialization: A plus sign (+) in front of a command means that command is executed when the key is pressed down, the minus sign (-) in front of a command means that command is executed when the key is released.

The script begins with the system linked to `grnrt`**`0`**. This executes `grst`**`1`** and executes an “echo FULL RESET of Grenade Slots”. The command `grst`**`1`** links `grnslt`**`a`** through `grnslt`**`d`** each incrementally to `grnrt`**`a1`** through `grnrt`**`d1`** setting up the selection of grenades. The `gsrst` command, the `gbrst` command, and `skiprst` command are executed.

The `gsrst` command links the the grenade slot place holders `+gs`**`1`** to `+gs`**`4`** to the null command with their release action of `-gs`**`1`** to `-gs`**`4`** to `gbutrot`. The `gbutrot` command causes the rotation of the grenades to be thrown.

The `gbrst` command links `gbutrot` to the first grenade throw slot linker, gbutrot**1** and `+gtoss` is set to `+gtall` and `-gtoss` is set to `-gtall` to default throw any grenade you have.

The command `skprst` links `skip`**`1`** through `skip`**`4`** to `skip`**`1`**`rst` through `skip`**`4`**`rst`

The commands `gbutrot`**`1`** through `gbutrot`**`4`** increment the current `+gtoss` command to `+gs`**`1`** through `+gs`**`4`** slot placeholders and `-gtoss` command to `-gs`**`1`** through `-gs`**`4`** slot placeholders, with `gbutrot` linked to the next `gbutrot`**`2`** through `gbutrot`**`4`** with a `skip`**`1`** through `skip`**`4`** to jump over empty slots as the user is throwing or halt rotation for a selected grenade throw.

The commands `skip`**`1`**`rst` though `skip`**`4`**`rst` work with `gbutrot`**`1`** through `gbutrot`**`4`** incrementally with the `skip`**`1`** through `skip`**`4`** commands to rotate grenades for throwing. The commands `skip`**`1`**`rst` through `skip`**`4`**`rst` incrementally set the `+gs`**`1`** through `+gs`**`4`** slot placeholders for grenades to `+gtoss` and `-gtoss`, then sets `gbutrot` to the next `gbutrot`**`3`** or `gbutrot`**`4`** or `skip`**`4`**`rst`, and then executes `skip`**`2`** through `skip`**`4`** to skip empty slots or halt at that slot for throwing. The command `skip`**`4`**`rst` resets everything to throw any grenade you have.

Grenade Selection:

The commands grnslt**a1** through grnslt**d4** incrementally execute grst**2** through grst**5** incrementing the commands grnslt**a** through grnslt**d** connected to operator keys to only use each type of grenade once. The commands grnslt**a1** through grnslt**d4** also link the +gs**1** and -gs**1** through +gs**4** and -gs**4** commands to which specific type of grenade to throw; +gthe and -gthe (high explosive), +gtfb and -gtfb (flash bang), +gttg and -gttg (tear gas), or +gtsmk and -gtsmk (smoke grenade). The commands grnslt**a1** through grnslt**d4** also link the corresponding skip**n** to execute the command null, which stops the execution of the loop.

First the V key is pressed to put a high explosive in slot 3 through executing grnslt**c** which executes grnslt**c1**. This links +gthe and -gthe to +gs**3** and -gs**3**, and skip**3** to null so the script stops on the third slot to throw the grenade. The command echo Grenade Slot 3 high explosive grenade is executed.

The X key is pressed once repeating the execution for high explosive, to put +gtfb and -gtfb in +gs**1** and -gs**1** and set skip**1** to null so the script stops on the first slot to throw a grenade. The X key is pressed again to put +gttg and -gttg in +gs**1** and -gs**1**. Once more, press the X key to put +gtsmk and -gtsmk in +gs**1** and -gs**1**.

The system is now set to throw a smoke grenade first, skip the second slot, throw the high explosive second, skip the fourth and last slot, then go to throwing any grenade you might have.

Throwing Mechanics:

The command gbutrot executes gbutrot**1** through gbutrot**4**, which incrementally link to the grenade slots +gs**1** through +gs**4** and execute a corresponding skip**1** through skip**4.**

The Mouse2 key is pressed executing +gtoss which executes +gs**1** which executes +gtsmk. The Mouse2 key is released, -gs**1** executes gbutrot which executes gbutrot**2** to set up the next Mouse2 press. The commands +gs**2** and -gs**2** are skipped to the third slot executing skip**2** which executes skip**2**rst. The Mouse2 key is pressed executing +gtoss which executes +gs**3** which executes +gthe. When the Mouse2 key is released, -gs**3** executes gbutrot which executes gbutrot**4**. The command +gs**4** and -gs**4** are skipped by executing skip**4** which executes skip**4**rst that sets the Mouse2 grenade throwing to throw any grenade available.

With each toss it executes echo and the grenade thrown.

Start of grenade-combination_lock.cfg

```
//bind keys to commands for user input
bind x grnslta //bind the X key to grenade slot 1
bind c grnsltb //bind the C key to grenade slot 2
bind v grnsltc //bind the V key to grenade slot 3
bind b grnsltd //bind the B key to grenade slot 4
bind n grnrt0 //bind the N key to reset the grenade slots
bind MOUSE2 +gtoss //bind mouse2 to the action of throwing the grenades

// Grenade Throwing System
alias null ""
alias gsrst "alias +gs1 null;alias -gs1 gbutrot;alias +gs2 null;alias -gs2 gbutrot;alias +gs3 null;alias -gs3 gbutrot;alias +gs4 null;alias -gs4 gbutrot"
alias gbrst "alias gbutrot gbutrot1;alias +gtoss +gtall;alias -gtoss -gtall"
alias skiprst "alias skip1 skip1rst;alias skip2 skip2rst;alias skip3 skip3rst;alias skip4 skip4rst"
```

```
alias skip1rst "alias +gtoss +gs2;alias -gtoss -gs2;alias gbutrot gbutrot3;skip2"
alias skip2rst "alias +gtoss +gs3;alias -gtoss -gs3;alias gbutrot gbutrot4;skip3"
alias skip3rst "alias +gtoss +gs4;alias -gtoss -gs4;alias gbutrot skip4rst;skip4"
alias skip4rst "alias +gtoss +gtall;alias -gtoss -gtall;alias gbutrot null"

alias grst1 "alias grnslta grnrta1;alias grnsltb grnrtb1;alias grnsltc grnrtc1;alias grnsltd grnrtd1;gsrst;gbrst;skiprst"
alias grst2 "alias grnslta grnrta2;alias grnsltb grnrtb2;alias grnsltc grnrtc2;alias grnsltd grnrtd2"
alias grst3 "alias grnslta grnrta3;alias grnsltb grnrtb3;alias grnsltc grnrtc3;alias grnsltd grnrtd3"
alias grst4 "alias grnslta grnrta4;alias grnsltb grnrtb4;alias grnsltc grnrtc4;alias grnsltd grnrtd4"
alias grst5 "alias grnslta grnrt0;alias grnsltb grnrt0;alias grnsltc grnrt0;alias grnsltd grnrt0"

alias grnrt0 "grst1;echo FULL RESET of Grenade Slots"
alias grnrta1 "grst2;alias +gs1 +gthe;alias -gs1 -gthe;alias skip1 null;echo Grenade Slot 1 high explosive grenade"
alias grnrta2 "grst3;alias +gs1 +gtfb;alias -gs1 -gtfb;alias skip1 null;echo Grenade Slot 1 flashbang grenade"
alias grnrta3 "grst4;alias +gs1 +gttg;alias -gs1 -gttg;alias skip1 null;echo Grenade Slot 1 teargas grenade"
alias grnrta4 "grst5;alias +gs1 +gtsmk;alias -gs1 -gtsmk;alias skip1 null;echo Grenade Slot 1 smoke grenade"
alias grnrtb1 "grst2;alias +gs2 +gthe;alias -gs2 -gthe;alias skip2 null;echo Grenade Slot 2 high explosive grenade"
alias grnrtb2 "grst3;alias +gs2 +gtfb;alias -gs2 -gtfb;alias skip2 null;echo Grenade Slot 2 flashbang grenade"
alias grnrtb3 "grst4;alias +gs2 +gttg;alias -gs2 -gttg;alias skip2 null;echo Grenade Slot 2 teargas grenade"
alias grnrtb4 "grst5;alias +gs2 +gtsmk;alias -gs2 -gtsmk;alias skip2 null;echo Grenade Slot 2 smoke grenade"
alias grnrtc1 "grst2;alias +gs3 +gthe;alias -gs3 -gthe;alias skip3 null;echo Grenade Slot 3 high explosive grenade"
alias grnrtc2 "grst3;alias +gs3 +gtfb;alias -gs3 -gtfb;alias skip3 null;echo Grenade Slot 3 flashbang grenade"
alias grnrtc3 "grst4;alias +gs3 +gttg;alias -gs3 -gttg;alias skip3 null;echo Grenade Slot 3 teargas grenade"
alias grnrtc4 "grst5;alias +gs3 +gtsmk;alias -gs3 -gtsmk;alias skip3 null;echo Grenade Slot 3 smoke grenade"
alias grnrtd1 "grst2;alias +gs4 +gthe;alias -gs4 -gthe;alias skip4 null;echo Grenade Slot 4 high explosive grenade"
alias grnrtd2 "grst3;alias +gs4 +gtfb;alias -gs4 -gtfb;alias skip4 null;echo Grenade Slot 4 flashbang grenade"
alias grnrtd3 "grst4;alias +gs4 +gttg;alias -gs4 -gttg;alias skip4 null;echo Grenade Slot 4 teargas grenade"
alias grnrtd4 "grst5;alias +gs4 +gtsmk;alias -gs4 -gtsmk;alias skip4 null;echo Grenade Slot 4 smoke grenade"

alias gbutrot1 "alias +gtoss +gs1;alias -gtoss -gs1;alias gbutrot gbutrot2;skip1"
alias gbutrot2 "alias +gtoss +gs2;alias -gtoss -gs2;alias gbutrot gbutrot3;skip2"
alias gbutrot3 "alias +gtoss +gs3;alias -gtoss -gs3;alias gbutrot gbutrot4;skip3"
alias gbutrot4 "alias +gtoss +gs4;alias -gtoss -gs4;alias gbutrot skip4rst;skip4"

alias +gthe "echo throw high explosive grenade"
alias -gthe "gbutrot"
alias +gtfb "echo throw flashbang grenade"
alias -gtfb "gbutrot"
alias +gttg "echo throw teargas grenade"
alias -gttg "gbutrot"
alias +gtsmk "echo throw smoke grenade"
alias -gtsmk "gbutrot"
alias +gtall "echo throw any available grenade"
alias -gtall "null"

//initialize the system
grnrt0
```

End of grenade-combination_lock.cfg